EXPERIMENT 2: CLIENT/SERVER USING RPC/RMI

Date Of Performance:
Date Of Submission:

1. Aim:

To design and implement a Client-Server application using RPC/RMI, where a client invokes remote methods on a server as if they were local, enabling distributed communication, data exchange, and service execution across a network in a transparent and efficient manner

Objectives

1. To understand the concept of Client–Server architecture in distributed systems.
2. To study the working of Remote Procedure Call (RPC) / Remote Method Invocation (RMI).
3. To implement communication between client and server processes over a network.
4. To enable a client to invoke methods on a remote server as if they were local.
5. To demonstrate inter-process communication and data exchange using RPC/RMI.
6. To understand stub and skeleton mechanisms used in remote communication.
7. To handle request–response interaction between distributed components.
8. To analyze the advantages of RPC/RMI in building distributed applications.

Scope of Client/Server using RPC/RMI

1. Understanding Client–Server Architecture and RPC/RMI Communication
The scope of studying Client–Server architecture with RPC/RMI includes understanding how distributed systems are structured and how different components communicate over a network. In a client–server model, the client requests services while the server processes those requests and returns the results. RPC and RMI provide mechanisms that allow remote communication between these components, making it possible for programs to execute procedures or methods located on another system. This helps in building distributed applications where resources and services are shared efficiently across multiple machines.
2. Implementation of Network Communication between Client and Server
The scope also includes implementing practical communication between client and server processes through network programming. Using RPC or RMI, the client can invoke methods or procedures on a remote server as if they were local functions. This simplifies application development because the underlying network communication is handled by the system itself. Such implementations are widely used in distributed systems where different applications interact and exchange information across networks.
3. Inter-Process Communication and Stub Mechanism
Another important scope is understanding how inter-process communication occurs between distributed components using RPC/RMI mechanisms. In this process, **stubs and skeletons** play a vital role by acting as intermediaries between the client and the server. The client stub converts procedure calls into network requests, while the server skeleton receives the request and executes the required procedure. This mechanism hides the complexity of network communication and allows seamless interaction between distributed processes.
4. Request–Response Interaction and Advantages in Distributed Applications
The scope also covers analyzing how request–response interactions are handled in distributed systems and understanding the advantages of RPC/RMI in building scalable applications. These technologies support efficient communication between multiple clients and servers, enabling reliable data exchange and

resource sharing. They are widely used in enterprise systems, cloud computing environments, and large-scale distributed applications where modular design and efficient communication are essential.

2. Literature Survey:

2.1 Theory:

1. Remote Procedure Call (RPC):

Remote Procedure Call (RPC) is one of the most successful middleware models used in modern distributed applications for communication.

Definition- RPC allows a program to call a procedure that resides on a remote machine using a local procedure call. The communication between the client and the server is hidden from the programmer.

Working of RPC:-

1. The client invokes a procedure as if it were a local call.
2. The client stub sends the request to the remote server.
3. The server executes the procedure and returns the result.
4. The client receives the result transparently.

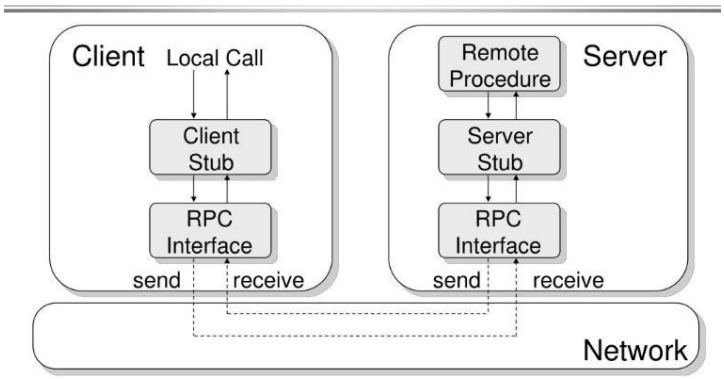


Fig.1 Remote Procedure Call(RPC) Mechanism in Distributed System

2. Remote Method Invocation (RMI):

Remote Method Invocation (RMI) is a Java-based distributed object technology that allows objects to communicate with each other over a network. In RMI, objects are distributed across different systems, enabling a client to access and use remote objects as if they were local. These remote objects are located using the RMI Registry, which acts as a directory service where the server registers available objects and the client searches for them. Clients interact with remote objects through well-defined interfaces, ensuring structured and secure communication.

The working of RMI involves the server first registering remote objects with the RMI Registry so they can be discovered by clients. The client then looks up the required object using the registry and invokes methods on the remote object through the provided interfaces. This mechanism simplifies remote communication by hiding network complexities. However, a major disadvantage of RMI is that it works only on the Java platform and does not support heterogeneity, meaning it cannot easily communicate with systems built using different programming languages or technologies.

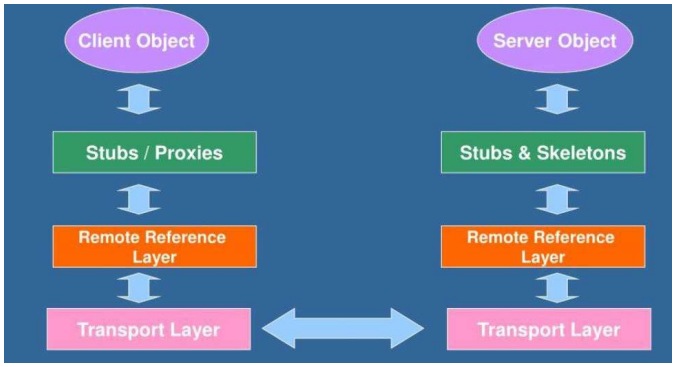
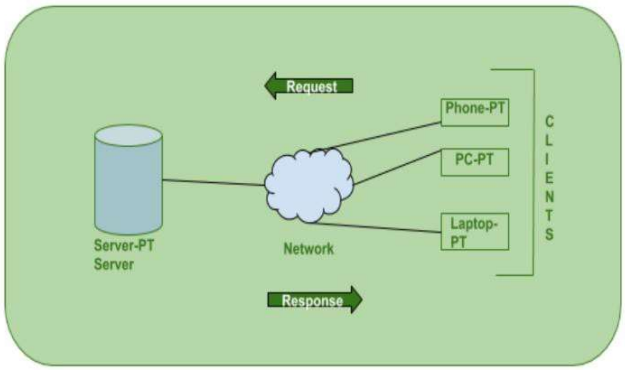


Fig.2 RMI Architecture

Client–Server Model

The Client–Server model is a widely used distributed application architecture in which responsibilities are divided between service providers called servers and service requesters known as clients. The server is a powerful system that stores data, manages resources, and provides services, while the client is a user-side system that requests those services over a network. In this model, the client sends a request to the server, such as asking for a webpage or accessing an email account, and the server processes the request by retrieving or computing the required information. After processing, the server sends the response back to the client in a structured format. Clients do not share their resources with each other; all communication takes place through the server, which ensures better control, security, and data management. This architecture is commonly used in many real-world applications such as email systems, the World Wide Web, online banking, and cloud-based services, making it a fundamental model in modern networking and distributed computing.

Client-Server Model:



Client and Server Description:

Client

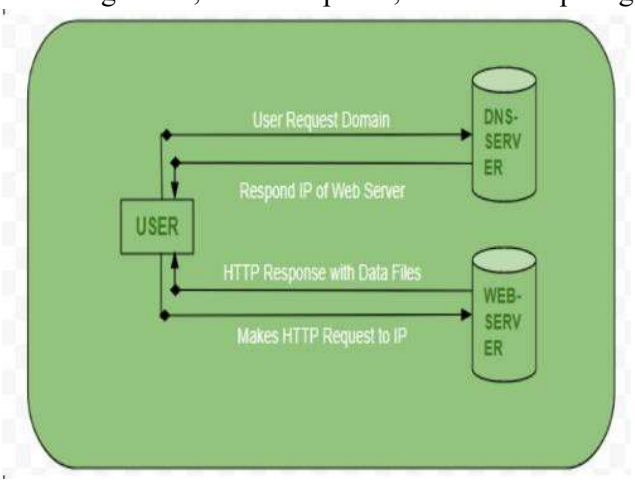
A client is a computer system or software application that initiates requests for services or resources over a network. It interacts with the user and sends requests to the server for processing or data retrieval. The client depends on the server to perform operations and return the required results, such as web pages, files, or services.

Server

A server is a powerful computer system or software application that **receives and processes requests** from multiple clients simultaneously. It stores data, manages resources, and provides services such as file access, database operations, or web hosting. The server responds to client requests and ensures reliable and secure service delivery.

How the Browser Interacts with Servers

1. User enters a URL in the browser
2. Browser sends a request to the DNS server
3. DNS server returns the IP address of the web server
4. Browser sends an HTTP/HTTPS request to the web server
5. Server responds with the requested files
6. Browser renders the website using DOM, CSS interpreter, and JavaScript engine



2.2 Algorithm:

2.2.1 Remote Procedure Call (RPC):

INPUT: Procedure name P, parameters x1, x2, ... xn
OUTPUT: Result R
BEGIN
1. Client invokes procedure P(x1, x2, ... xn)
2. Client_Stub:
   Marshal parameters into message M
   M ← Encode(P, x1, x2, ... xn)
3. Send message M to Server through network
4. Server_Stub:
   Receive message M
   (P, x1, x2, ... xn) ← Decode(M)
5. Execute procedure on Server
   R ← Execute(P, x1, x2, ... xn)
6. Server_Stub:
   Response ← Encode(R)
7. Send Response back to Client
8. Client_Stub:
   R ← Decode(Response)
9. Return result R to Client Program
END

2.2.2 Remote Method Invocation (RMI):

INPUT: Method name M, parameters p1, p2, ... pn

OUTPUT: Result R
BEGIN
1. Define Remote_Interface containing method M
2. Server:
   Implement Remote_Interface
   Create Remote_Object
3. Register Remote_Object in RMI_Registry
4. Client:
   Connect to RMI_Registry
   RemoteRef ← Lookup(Remote_Object)
5. Client invokes method
   R ← RemoteRef.M(p1, p2, ... pn)
6. Server:
   Execute method M on Remote_Object
   R ← Process(p1, p2, ... pn)
7. Return result R to Client
8. Client receives result R
END

2.2.3 Client–Server Model

INPUT: Client request Req
OUTPUT: Server response Res
BEGIN
SERVER SIDE
1. Create socket S
2. Bind S to Port_Number
3. Listen for incoming connections
CLIENT SIDE
4. Create socket C
5. Connect C to Server_IP and Port_Number
6. Send request Req to Server
SERVER SIDE
7. Accept connection from Client
8. Receive Req
9. Process Req
   Res ← Process(Req)
10. Send response Res to Client
CLIENT SIDE
11. Receive Res
12. Display Res
13. Close Client socket
14. Close Server socket
END

2.2 Programming Language used:

2.2.1.1 Libraries Used

1. socket
Purpose: Provides a low-level networking interface for creating client and server connections using TCP/IP communication.

Syntax: socket.socket(socket.AF_INET, socket.SOCK_STREAM)

Usage in Program: Used to create both client and server sockets for communication.

2. threading
Purpose: Allows the server to run concurrently in a separate thread to handle client requests.

Syntax: threading.Thread(target=function_name).start()

Usage in Program: Used to run the RPC server in a background thread.

3. struct
Purpose: Used for packing and unpacking binary data before transmitting it over the network.

Syntax: struct.pack(format, values)
struct.unpack(format, data)

Usage in Program: Used to convert integers into binary format before sending them through the socket.

4. time
Purpose: Provides functions for time-related operations such as delays.

Syntax: time.sleep(seconds)

Usage in Program: Ensures the server starts before the client attempts to connect.

2.2.1.2 Functions / Methods Used

1. socket()
Purpose: Creates a new socket object for communication.

Syntax: socket.socket(socket.AF_INET, socket.SOCK_STREAM)

2. bind()
Purpose: Associates the server socket with an IP address and port number.

Syntax: socket.bind((IP_address, port_number))

Example: server_socket.bind(("127.0.0.1", 5000))

3. listen()
Purpose: Puts the server socket into listening mode to wait for client requests.

Syntax: socket.listen(backlog)

Example: server_socket.listen(1)

4. accept()
Purpose: Accepts incoming client connection requests.

Syntax: connection, address = socket.accept()

5. connect()
Purpose: Establishes a connection between the client and the server.

Syntax: socket.connect((IP_address, port_number))

Example: client_socket.connect(("127.0.0.1", 5000))

6. send()
Purpose: Sends data from client to server or server to client.

Syntax: socket.send(data)

7. recv()
Purpose: Receives data from the network.

Syntax: socket.recv(buffer_size)

Example: data = conn.recv(8)

8. struct.pack()
Purpose: Converts data into binary format before sending.

Syntax: struct.pack("format", values)

Example: struct.pack("ii", a, b)

9. struct.unpack()
Purpose: Converts received binary data back into usable values.

Syntax: struct.unpack("format", data)

Example: a, b = struct.unpack("ii", data)

10. close()
Purpose: Closes the socket connection after communication is completed.

Syntax: socket.close()

2.2.2 Java Programming Language (For RMI Implementation)

The Remote Method Invocation (RMI) experiment is implemented using the **Java programming language**, which provides built-in support for **distributed object communication**.

Java RMI allows a program to **invoke methods of an object located on another machine** as if the object were local. It uses **remote interfaces, server implementations, and the RMI registry** to enable communication between client and server applications.

2.2.2.1 Packages Used

1. java.rmi.Remote
Purpose: Defines remote interfaces for method invocation.

Syntax: public interface Calculator extends Remote

2. java.rmi.RemoteException
Purpose: Handles communication-related exceptions during remote calls.

Syntax: throws RemoteException

3. java.rmi.server.UnicastRemoteObject
Purpose: Exports remote objects to make them accessible to clients.

Syntax: class CalculatorImpl extends UnicastRemoteObject

4. java.rmi.Naming
Purpose: Used for binding and locating remote objects in the RMI registry.

Syntax: Naming.rebind("ServiceName", object);
Naming.lookup("rmi://localhost/ServiceName");

5. java.rmi.registry.LocateRegistry
Purpose: Used to create and manage the RMI registry service.

Syntax: LocateRegistry.createRegistry(1099);

2.2.2.2 Methods / Classes Used

1. add()
Purpose: Remote method defined in the interface and implemented on the server.

Syntax: int add(int a, int b) throws RemoteException;

2. rebind()
Purpose: Registers the remote object with the RMI registry.

Syntax: Naming.rebind("ServiceName", object);

3. lookup()
Purpose: Allows the client to locate the remote object.

Syntax: Naming.lookup("rmi://localhost/ServiceName");

4. createRegistry()
Purpose: Starts the RMI registry service on a specific port.

Syntax: LocateRegistry.createRegistry(1099);

5. extends UnicastRemoteObject
Purpose: Makes the server object available for remote access.

Syntax: class ClassName extends UnicastRemoteObject

6. implements Remote
Purpose: Marks the interface for remote method invocation.

Syntax: public interface Calculator extends Remote

3. Implementation

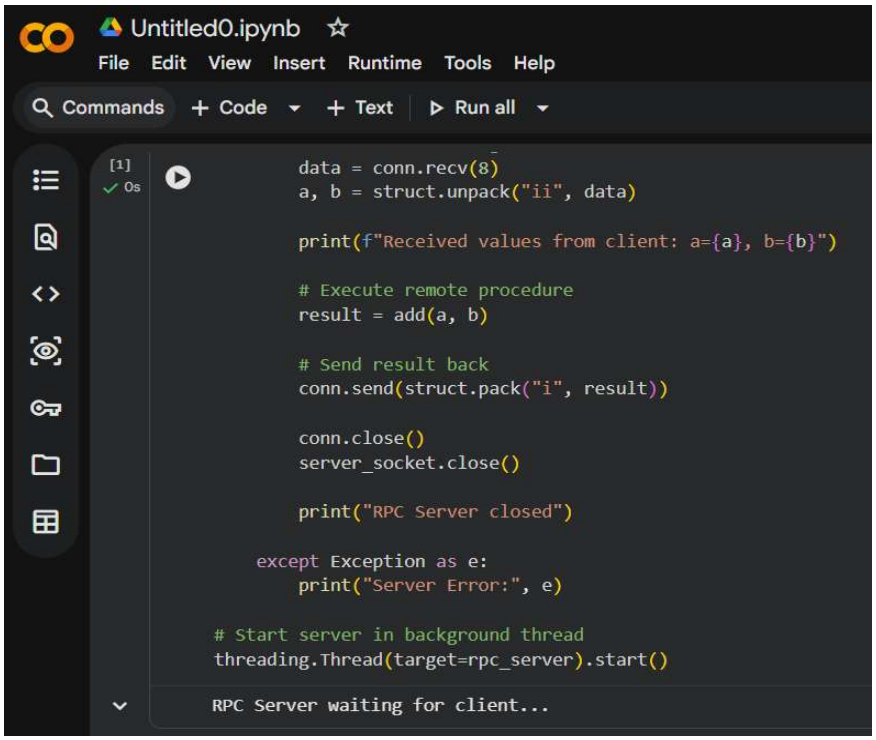
3.1 Remote Procedure Call (RPC):

Server side:

```
import socket
import threading
import struct
# Remote Procedure
def add(a, b):
    return a + b
def rpc_server():
    try:
        server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        server_socket.bind(("127.0.0.1", 5000))
        server_socket.listen(1)
        print("RPC Server waiting for client...")

        conn, addr = server_socket.accept()
        print("Client connected:", addr)

        # Receive two integers from client
        data = conn.recv(8)
        a, b = struct.unpack("ii", data)
        print(f"Received values from client: a={a}, b={b}")

        # Execute remote procedure
        result = add(a, b)
        # Send result back
        conn.send(struct.pack("i", result))
        conn.close()
        server_socket.close()
        print("RPC Server closed")

    except Exception as e:
        print("Server Error:", e)

# Start server in background thread
threading.Thread(target=rpc_server).start()
```

Command:- run
Platform:- Google Colab

Output:



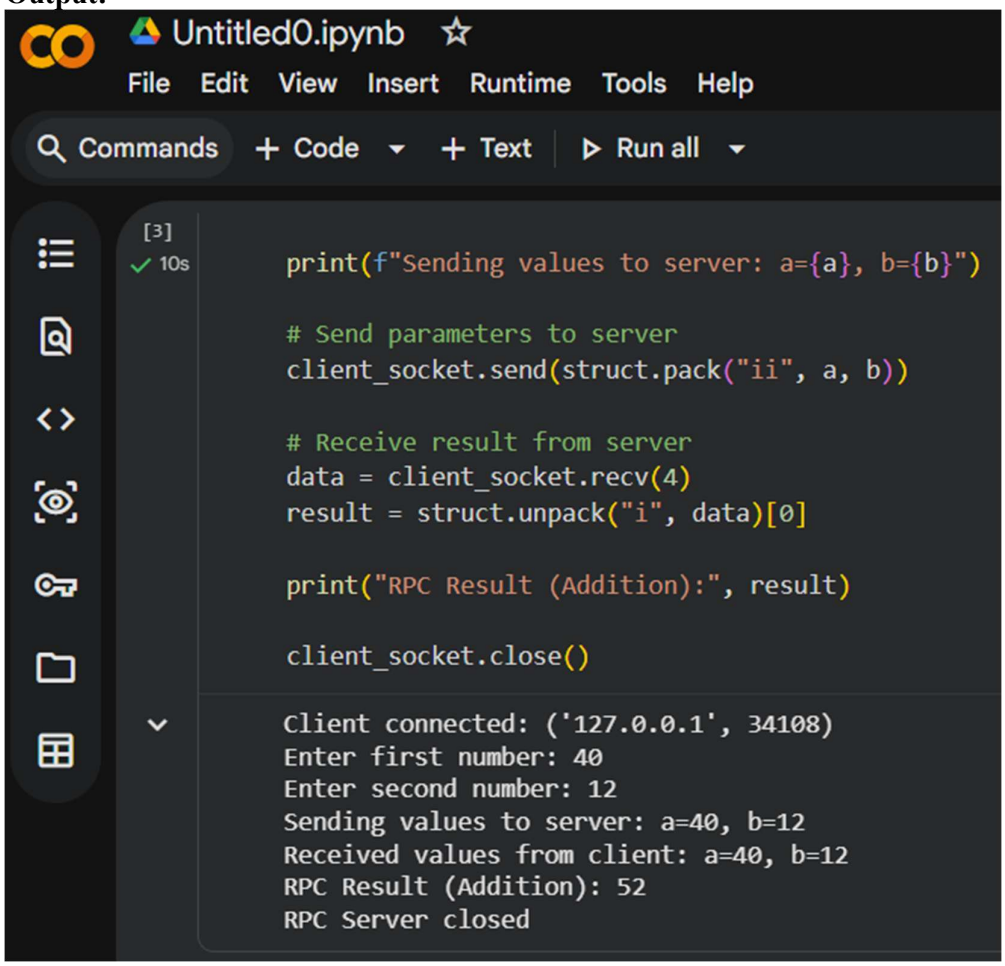
Client Side:

```
import socket
import struct
import time

time.sleep(1)  # Ensure server is ready
client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client_socket.connect(("127.0.0.1", 5000))

# USER INPUT
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
print(f"Sending values to server: a={a}, b={b}")

# Send parameters to server
client_socket.send(struct.pack("ii", a, b))

# Receive result from server
data = client_socket.recv(4)
result = struct.unpack("i", data)[0]
print("RPC Result (Addition):", result)
client_socket.close()
```

Command:- run

Platform:- Google Colab

Output:



3.2 Remote Method Invocation (RMI):

3.2.1 Calculator.java

```
import java.rmi.Remote;
import java.rmi.RemoteException;
// Remote interface
public interface Calculator extends Remote {
    int add(int a, int b) throws RemoteException;
}
```

3.2.2 CalculatorAdd.java

```
import java.rmi.server.UnicastRemoteObject;
import java.rmi.RemoteException;
// Implements the remote interface
public class CalculatorImpl extends UnicastRemoteObject implements Calculator {
    protected CalculatorImpl() throws RemoteException {
        super();
    }
    // Method implementation
```

```
  public int add(int a, int b) throws RemoteException {
        return a + b;
    }
}
```

3.2.3 RMIServer.java

```
import java.rmi.Naming;
import java.rmi.registry.LocateRegistry;

public class RMIServer {
    public static void main(String[] args) {
        try {
            LocateRegistry.createRegistry(1099); // AUTO registry

            AdderImpl obj = new AdderImpl();
            Naming.rebind("AdderService", obj);

            System.out.println("RMI Server started...");
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

3.2.4 RMIClient.java

```
import java.rmi.Naming;
public class RMIClient {
    public static void main(String[] args) {
        try {
            Adder obj = (Adder) Naming.lookup("rmi://localhost/AdderService");
            System.out.println("Result = " + obj.add(10, 20));
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

OUTPUT:

Command:- 1.javac *.java
2. java RMIServer

```
D:\Vaibhav>javac *.java

D:\Vaibhav>java Server
RMI Server is running...
```

```
D:\Vaibhav>java Client
Message from Server: Hello from RMI Server!
```

Conclusion

The Client–Server application using Remote Method Invocation (RMI) was successfully implemented and executed. RMI enables a client to invoke methods on objects located on a remote server as if they were local, thereby simplifying distributed application development. The communication details between client and server are completely hidden from the programmer, making the system easy to design and maintain. This implementation demonstrates how distributed objects can be registered, located, and accessed using the RMI registry. The experiment also highlights the importance of proper port management and registry handling in distributed systems. Overall, RMI provides an efficient and reliable mechanism for building object-oriented distributed applications in Java.

Sign and Remark:

R1 (4 Marks)	R2 (4 Marks)	R3 (4 Marks)	R4 (3 Marks)	Total Marks (15 Marks)	Signature